JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file.attribute

Class FileTime

java.lang.Object
    java.nio.file.attribute.FileTime

All Implemented Interfaces:

Comparable<FileTime>

```
public final class FileTime
extends Object
implements Comparable<FileTime>
```

Represents the value of a file's time stamp attribute. For example, it may represent the time that the file was last `modified`, `accessed`, or `created`.

Instances of this class are immutable.

Since:

1.7

See Also:

`Files.setLastModifiedTime(java.nio.file.Path, java.nio.file.attribute.FileTime)`, `Files.getLastModifiedTime(java.nio.file.Path, java.nio.file.LinkOption...)`

Method Summary

Instance Methods | Concrete Methods

Modifier and Type	Method	Description

int	**compareTo**(**FileTime** other)	Compares the value of two FileTime objects for order.
boolean	**equals**(**Object** obj)	Tests this FileTime for equality with the given object.
static **FileTime**	**from**(**Instant** instant)	Returns a FileTime representing the same point of time value on the time-line as the provided Instant object.
static **FileTime**	**from**(long value, **TimeUnit** unit)	Returns a FileTime representing a value at the given unit of granularity.
static **FileTime**	**fromMillis**(long value)	Returns a FileTime representing the given value in milliseconds.
int	**hashCode**()	Computes a hash code for this file time.
long	**to**(**TimeUnit** unit)	Returns the value at the given unit of granularity.
Instant	**toInstant**()	Converts this FileTime object to an Instant.
long	**toMillis**()	Returns the value in milliseconds.
String	**toString**()	Returns the string representation of this FileTime.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Detail

from

```
public static FileTime from(long value,
                            TimeUnit unit)
```

Returns a `FileTime` representing a value at the given unit of granularity.

Parameters:

`value - the value since the epoch (1970-01-01T00:00:00Z); can be negative`

`unit - the unit of granularity to interpret the value`

Returns:

`a FileTime representing the given value`

fromMillis

```
public static FileTime fromMillis(long value)
```

Returns a `FileTime` representing the given value in milliseconds.

Parameters:

`value - the value, in milliseconds, since the epoch (1970-01-01T00:00:00Z); can be negative`

Returns:

`a FileTime representing the given value`

from

```
public static FileTime from(Instant instant)
```

Returns a `FileTime` representing the same point of time value on the time-line as the provided `Instant` object.

Parameters:

`instant - the instant to convert`

Returns:

`a FileTime representing the same point on the time-line as the provided instant`

Since:

`1.8`

to

```
public long to(TimeUnit unit)
```

Returns the value at the given unit of granularity.

Conversion from a coarser granularity that would numerically overflow saturate to `Long.MIN_VALUE` if negative or `Long.MAX_VALUE` if positive.

Parameters:

`unit - the unit of granularity for the return value`

Returns:

`value in the given unit of granularity, since the epoch since the epoch (1970-01-01T00:00:00Z); can be negative`

toMillis

```
public long toMillis()
```

Returns the value in milliseconds.

Conversion from a coarser granularity that would numerically overflow saturate to `Long.MIN_VALUE` if negative or `Long.MAX_VALUE` if positive.

Returns:

`the value in milliseconds, since the epoch (1970-01-01T00:00:00Z)`

toInstant

```
public Instant toInstant()
```

Converts this `FileTime` object to an `Instant`.

The conversion creates an `Instant` that represents the same point on the time-line as this `FileTime`.

`FileTime` can store points on the time-line further in the future and further in the past than `Instant`. Conversion from such further time points saturates to `Instant.MIN` if earlier than `Instant.MIN` or `Instant.MAX` if later than `Instant.MAX`.

Returns:

an instant representing the same point on the time-line as this FileTime object

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Tests this `FileTime` for equality with the given object.

The result is `true` if and only if the argument is not `null` and is a `FileTime` that represents the same time. This method satisfies the general contract of the `Object.equals` method.

Overrides:

`equals` in class `Object`

Parameters:

obj - the object to compare with

Returns:

true if, and only if, the given object is a FileTime that represents the same time

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Computes a hash code for this file time.

The hash code is based upon the value represented, and satisfies the general contract of the `Object.hashCode()` method.

Overrides:

`hashCode` in class `Object`

Returns:

the hash-code value

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

compareTo

```
public int compareTo(FileTime other)
```

Compares the value of two `FileTime` objects for order.

Specified by:

`compareTo` in interface `Comparable<FileTime>`

Parameters:

other - the other FileTime to be compared

Returns:

0 if this FileTime is equal to other, a value less than 0 if this FileTime represents a time that is before other, and a value greater than 0 if this FileTime represents a time that is after other

toString

```
public String toString()
```

Returns the string representation of this `FileTime`. The string is returned in the ISO 8601 format:

```
YYYY-MM-DDThh:mm:ss[.s+]Z
```

where "[.s+]" represents a dot followed by one of more digits for the decimal fraction of a second. It is only present when the decimal fraction of a second is not zero. For example, `FileTime.fromMillis(1234567890000L).toString()` yields "2009-02-13T23:31:30Z", and `FileTime.fromMillis(1234567890123L).toString()` yields "2009-02-13T23:31:30.123Z".

A `FileTime` is primarily intended to represent the value of a file's time stamp. Where used to represent *extreme values*, where the year is less than "0001" or greater than "9999" then this method deviates from ISO 8601 in the same manner as the XML Schema language. That is, the year may be expanded to more than four digits and may be negative-signed. If more than four digits then leading zeros are not present. The year before "0001" is "-0001".

Overrides:

`toString` in class `Object`

Returns:

the string representation of this file time

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.